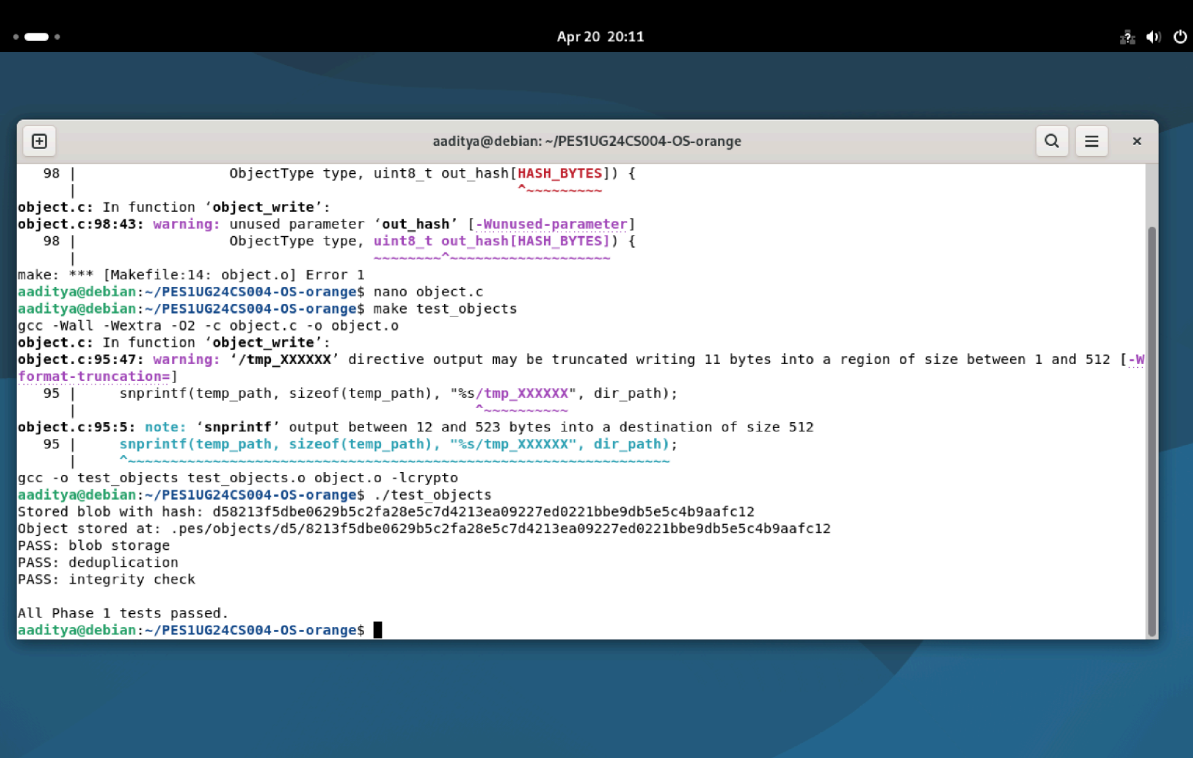


OS ORANGE PROBLEM SLOVING
AADITYA VASHISHT
PES1UG24CS004
4A

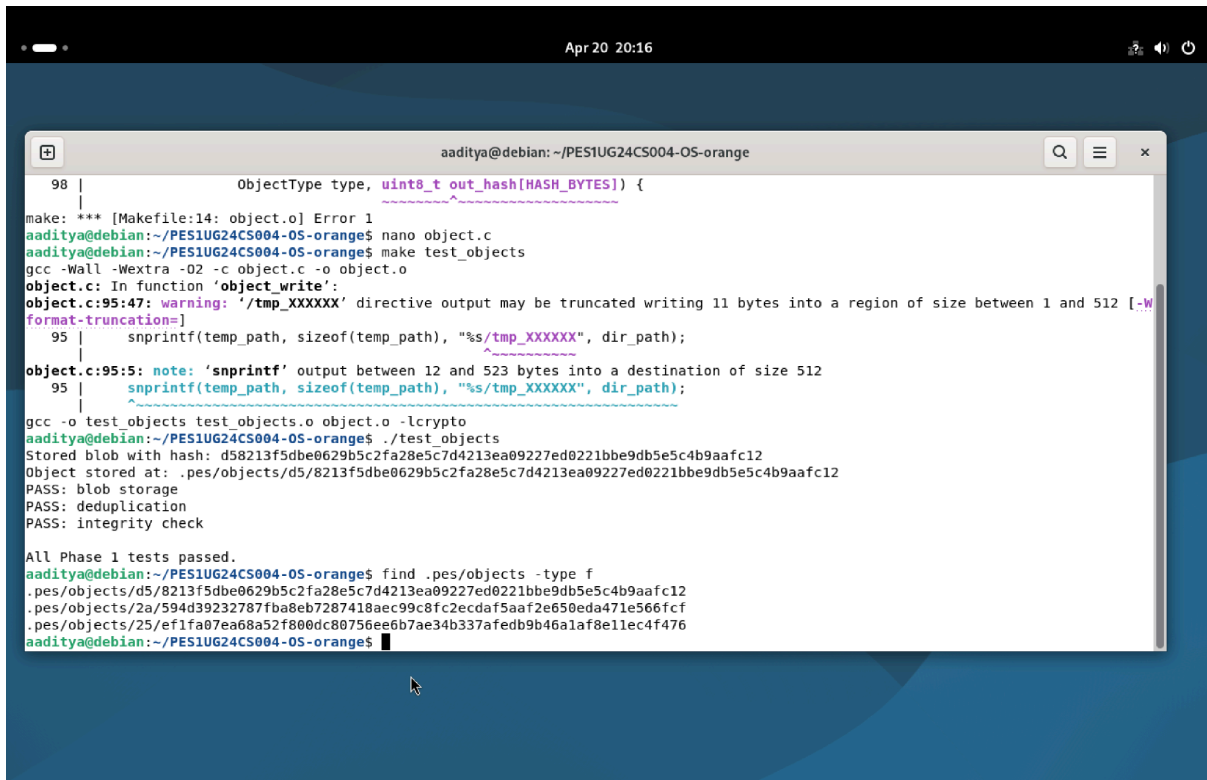
1A



```
aaditya@debian: ~/PES1UG24CS004-OS-orange
98 |         ObjectType type, uint8_t out_hash[HASH_BYTES]) {
object.c: In function 'object_write':
object.c:98:43: warning: unused parameter 'out_hash' [-Wunused-parameter]
98 |         ObjectType type, uint8_t out_hash[HASH_BYTES]) {
make: *** [Makefile:14: object.o] Error 1
aaditya@debian:~/PES1UG24CS004-OS-orange$ nano object.c
aaditya@debian:~/PES1UG24CS004-OS-orange$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_write':
object.c:95:47: warning: '/tmp_XXXXXX' directive output may be truncated writing 11 bytes into a region of size between 1 and 512 [-Wformat-truncation=]
95 |         snprintf(temp_path, sizeof(temp_path), "%s/tmp_XXXXXX", dir_path);
object.c:95:5: note: 'snprintf' output between 12 and 523 bytes into a destination of size 512
95 |         snprintf(temp_path, sizeof(temp_path), "%s/tmp_XXXXXX", dir_path);
gcc -o test_objects test_objects.o object.o -lcrypto
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
aaditya@debian:~/PES1UG24CS004-OS-orange$
```

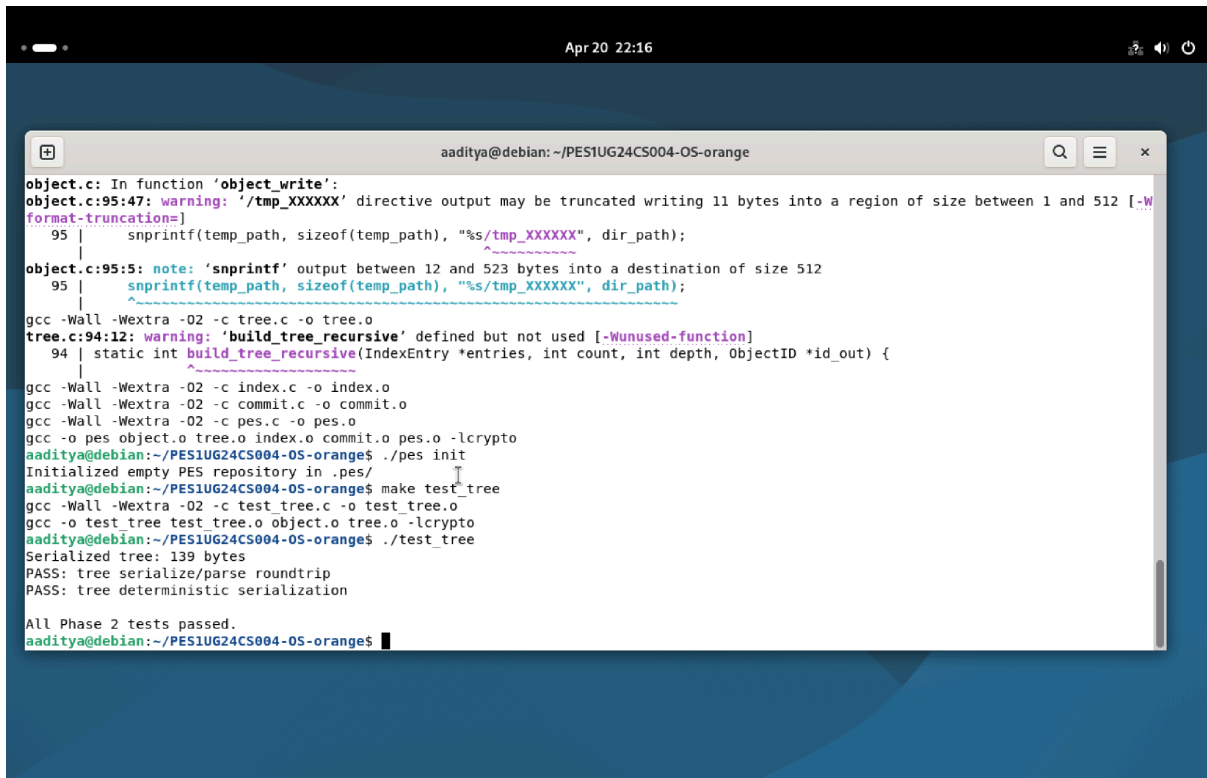
1B



```
aaditya@debian: ~/PES1UG24CS004-OS-orange
98 |         ObjectType type, uint8_t out_hash[HASH_BYTES]) {
make: *** [Makefile:14: object.o] Error 1
aaditya@debian:~/PES1UG24CS004-OS-orange$ nano object.c
aaditya@debian:~/PES1UG24CS004-OS-orange$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_write':
object.c:95:47: warning: '/tmp_XXXXXX' directive output may be truncated writing 11 bytes into a region of size between 1 and 512 [-Wformat-truncation=]
   95 |         snprintf(temp_path, sizeof(temp_path), "%s/tmp_XXXXXX", dir_path);
      |         ~~~~~^~~~~~
object.c:95:5: note: 'snprintf' output between 12 and 523 bytes into a destination of size 512
   95 |         snprintf(temp_path, sizeof(temp_path), "%s/tmp_XXXXXX", dir_path);
      |         ~~~~~^~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
aaditya@debian:~/PES1UG24CS004-OS-orange$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f80dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
aaditya@debian:~/PES1UG24CS004-OS-orange$
```

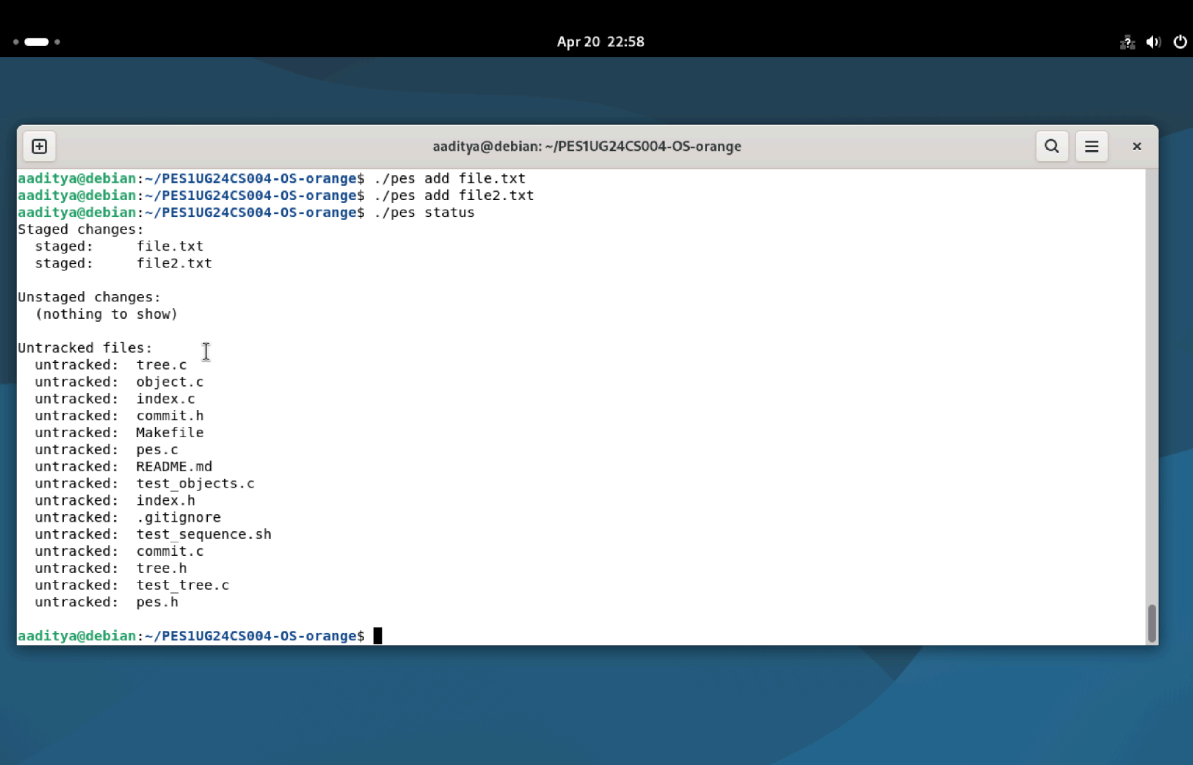
2A



```
aaditya@debian: ~/PES1UG24CS004-OS-orange
object.c: In function 'object_write':
object.c:95:47: warning: '/tmp_XXXXXX' directive output may be truncated writing 11 bytes into a region of size between 1 and 512 [-Wformat-truncation=]
   95 |         snprintf(temp_path, sizeof(temp_path), "%s/tmp_XXXXXX", dir_path);
      |         ~~~~~^~~~~~
object.c:95:5: note: 'snprintf' output between 12 and 523 bytes into a destination of size 512
   95 |         snprintf(temp_path, sizeof(temp_path), "%s/tmp_XXXXXX", dir_path);
      |         ~~~~~^~~~~~
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c:94:12: warning: 'build_tree_recursive' defined but not used [-Wunused-function]
   94 | static int build_tree_recursive(IndexEntry *entries, int count, int depth, ObjectID *id_out) {
      |            ^~~~~~
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes init
Initialized empty PES repository in .pes/
aaditya@debian:~/PES1UG24CS004-OS-orange$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
aaditya@debian:~/PES1UG24CS004-OS-orange$
```

3A



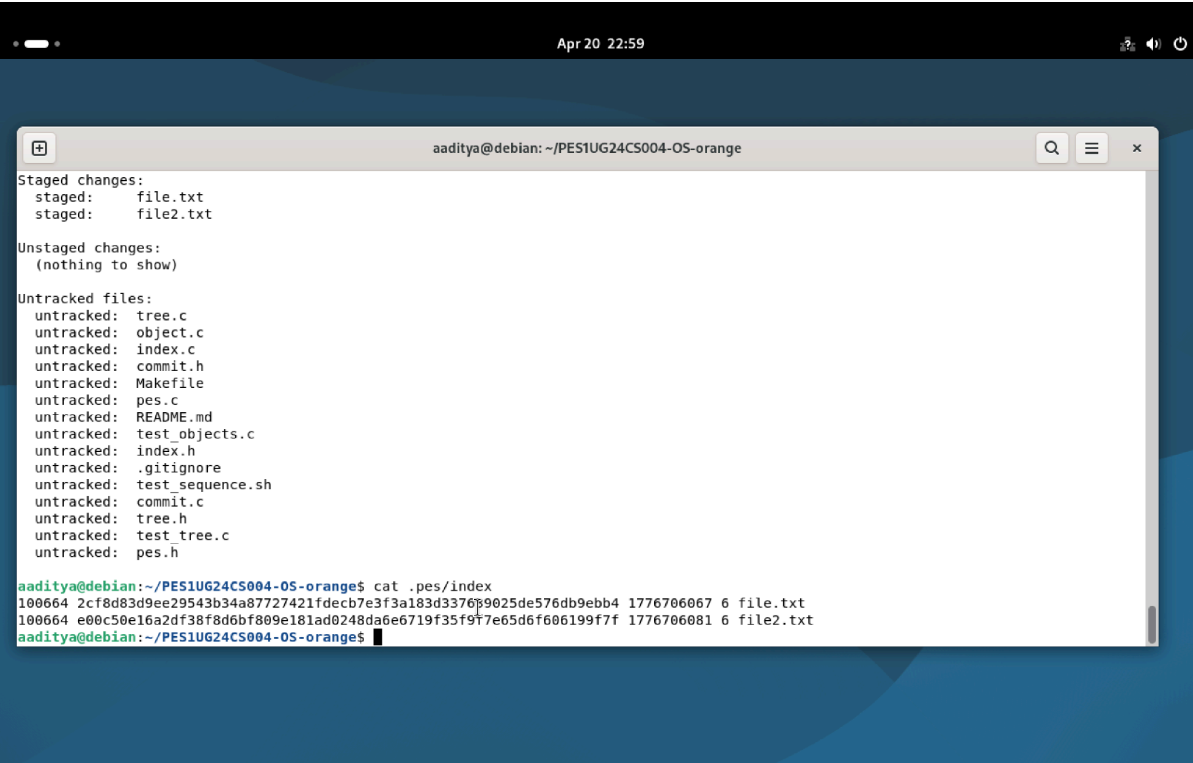
A terminal window titled 'aaditya@debian: ~/PES1UG24CS004-OS-orange' showing the output of the 'git status' command. The terminal has a dark blue background. The output shows two staged files (file.txt and file2.txt) and a list of untracked files including tree.c, object.c, index.c, commit.h, Makefile, pes.c, README.md, test_objects.c, index.h, .gitignore, test_sequence.sh, commit.c, tree.h, test_tree.c, and pes.h.

```
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes add file.txt
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes add file2.txt
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes status
Staged changes:
  staged:    file.txt
  staged:    file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked: tree.c
  untracked: object.c
  untracked: index.c
  untracked: commit.h
  untracked: Makefile
  untracked: pes.c
  untracked: README.md
  untracked: test_objects.c
  untracked: index.h
  untracked: .gitignore
  untracked: test_sequence.sh
  untracked: commit.c
  untracked: tree.h
  untracked: test_tree.c
  untracked: pes.h
aaditya@debian:~/PES1UG24CS004-OS-orange$
```

3B



A terminal window titled 'aaditya@debian: ~/PES1UG24CS004-OS-orange' showing the output of 'git status' followed by the command 'cat .pes/index'. The 'git status' output is identical to the previous screenshot. The 'cat' command output shows two lines of hexadecimal hashes followed by the filenames 'file.txt' and 'file2.txt'.

```
aaditya@debian:~/PES1UG24CS004-OS-orange$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776706067 6 file.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776706081 6 file2.txt
aaditya@debian:~/PES1UG24CS004-OS-orange$
```

4A

```

Apr 20 23:25

aaditya@debian: ~/PES1UG24CS004-OS-orange

gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes init
Initialized empty PES repository in .pes/
aaditya@debian:~/PES1UG24CS004-OS-orange$ echo "hello" > file.txt
aaditya@debian:~/PES1UG24CS004-OS-orange$ echo "world" > file2.txt
aaditya@debian:~/PES1UG24CS004-OS-orange$ touch .pes/index
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes add file.txt
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes add file2.txt
aaditya@debian:~/PES1UG24CS004-OS-orange$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776707520 6 file.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776707529 6 file2.txt
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes commit "first commit"
error: commit requires a message (-m "message")
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes commit -m "first commit"

[DEBUG] object_write reported SUCCESS
Committed: 237ca084bf09... first commit
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes log
commit 237ca084bf09ec43cb4b24222ed9b69760410382a6585f482dd2bf7fcac6a7b0
Author: PES User <pes@localhost>
Date: 1776707663

first commit00
aaditya@debian:~/PES1UG24CS004-OS-orange$

```

4B

```

Apr 20 23:26

aaditya@debian: ~/PES1UG24CS004-OS-orange

aaditya@debian:~/PES1UG24CS004-OS-orange$ touch .pes/index
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes add file.txt
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes add file2.txt
aaditya@debian:~/PES1UG24CS004-OS-orange$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776707520 6 file.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776707529 6 file2.txt
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes commit "first commit"
error: commit requires a message (-m "message")
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes commit -m "first commit"

[DEBUG] object_write reported SUCCESS
Committed: 237ca084bf09... first commit
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes log
commit 237ca084bf09ec43cb4b24222ed9b69760410382a6585f482dd2bf7fcac6a7b0
Author: PES User <pes@localhost>
Date: 1776707663

first commit00

aaditya@debian:~/PES1UG24CS004-OS-orange$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/23/7ca084bf09ec43cb4b24222ed9b69760410382a6585f482dd2bf7fcac6a7b0
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/d7/1b25693891507a11cd5dbc58f431b16b1ec9a907dafac66014e1585d775e
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/refs/heads/main
aaditya@debian:~/PES1UG24CS004-OS-orange$

```

```

aaditya@debian: ~/PES1UG24CS004-OS-orange
100664 2cf8d83d9ee29543b34a07727421fdec7e3f3a183d337639025de576db9ebb4 1776707520 6 file.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776707529 6 file2.txt
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes commit "first commit"
error: commit requires a message (-m "message")
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes commit -m "first commit"

[DEBUG] object_write reported SUCCESS
Committed: 237ca084bf09... first commit
aaditya@debian:~/PES1UG24CS004-OS-orange$ ./pes log
commit 237ca084bf09ec43cb4b24222ed9b69760410382a6585f482dd2bf7fcac6a7b0
Author: PES User <pes@localhost>
Date: 1776707663

    first commit00

aaditya@debian:~/PES1UG24CS004-OS-orange$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/23/7ca084bf09ec43cb4b24222ed9b69760410382a6585f482dd2bf7fcac6a7b0
.pes/objects/2c/f8d83d9ee29543b34a07727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/d7/1b25693891507a11cd5dbc58f431b16b1ec9a907dafac66014e1585d775e
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/refs/heads/main
aaditya@debian:~/PES1UG24CS004-OS-orange$ cat .pes/refs/heads/main
237ca084bf09ec43cb4b24222ed9b69760410382a6585f482dd2bf7fcac6a7b0
aaditya@debian:~/PES1UG24CS004-OS-orange$ cat .pes/HEAD
ref: refs/heads/main
aaditya@debian:~/PES1UG24CS004-OS-orange$

```

THEORY QUESTIONS

Phase 5: Branching and Checkout

Q5.1:

For switching branches, you update `.pes/HEAD` to point to the new branch file and overwrite `.pes/index` with that branch's tree state. Then, you sync your working directory by deleting files that aren't in the new branch and extracting the correct blobs from the object store. It's complex because we must avoid deleting "untracked" files that the user created but never staged.

Q5.2:

We compare the files on our disk with the versions recorded in the index using metadata like size and modified time. If they differ, you re-hash the file. If this hash doesn't match the index, the file is "dirty." The system should block a checkout if these local changes would be overwritten by the incoming branch, preventing accidental data loss.

Q5.3:

A detached HEAD happens when `.pes/HEAD` contains a specific commit hash instead of a branch name. You can still make commits, but they won't belong to any branch. If you switch to another branch, these "anonymous" commits

become orphaned. To recover them, you find the last commit hash in your log and manually create a new branch reference at that point.

Phase 6: Garbage Collection (GC)

Q6.1:

We use a Mark-and-Sweep algorithm. Starting from your branch tips (refs), you traverse every link—from commits to trees to blobs—and save every encountered hash in a Hash Set (the "reachable" list). Finally, you scan the .git/objects folder and delete any file whose hash isn't in your set. For 100k commits, deduplication ensures you only process unique objects, keeping the search efficient.

Q6.2:

The danger is a "race condition" where GC runs while you are mid-commit. For example, if `git add` creates a blob but `git commit` hasn't linked it to a tree yet, GC might think the blob is junk and delete it. To fix this, Git uses a grace period (usually 2 weeks). It checks the file's timestamp and refuses to delete anything created recently, even if it looks unreachable.